AGORA

멀티 에이전트 기반 AI 토론 플랫폼

기획 제안서

팀원 1: Ulmasov Asadbek Shukhratovich

팀원 2: Nuriddinov Bekhruzbek Abdumutaliv

팀원 3: Su Sandi Ko

목차

1. 프로젝트 개요

1.1 문제정의 및 솔루션

현재 대부분의 생성형 AI 서비스는 사용자의 질문에 단일 응답을 제시하는 방식으로 동작한다. 이 방식은 다음과 같은 두 가지 핵심 문제를 가진다.

첫째, Hallucination이다. 단일 AI는 잘못된 정보를 자신 있게 제시하더라도 이를 검증하거나 반박하는 주체가 없다. 오류가 그대로 사용자에게 전달된다.

둘째, 아부 편향(Sycophancy)이다. 사용자가 편향된 주장을 할 때 AI는 이에 동조하는 방향으로 답변을 생성하는 경향이 있다. 비판적 검토가 이루어지지 않는다.

이러한 문제는 단순 정보 탐색보다 다면적 사고가 필요한 상황, 즉 프로젝트 주제 선정, 전략 수립, 진로 결정과 같은 복잡한 의사결정에서 특히 크게 나타난다. 사용자가 다양한 관점을 얻으려면 여러 탭을 열어 각각 질문해야 하는데, 이는 번거롭고 결과를 체계적으로 비교하기도 어렵다.

솔루션: AGORA

AGORA는 여러 AI 에이전트가 서로 다른 역할과 관점을 바탕으로 하나의 질문을 분석하고 토론하는 Multi-Agent Decision Intelligence 플랫폼이다.

각 에이전트는 분석가(Analyst), 비평가(Critic), 전략가(Strategist), 개인 자문가(Personal Advisor)와 같은 역할을 부여받아 독립적으로 1차 의견을 제시한다. 이후 서로의 주장에 대해 반박, 보완, 확장하는 구조화된 토론 과정을 거친다. 마지막에는 Moderator AI가 전체 토론을 종합하여 최종 결론과 추천안을 제시한다.

결과적으로 사용자는 단일 답변이 아니라, 여러 관점이 교차 검증된 구조화된 의사결정 결과를 얻는다. 단순 답변에서 검증된 결론으로의 전환이 AGORA의 핵심 가치이다.

1.2 주 타겟 및 Stakeholder

AGORA의 주 타겟은 단일 정답보다 비교 가능한 여러 관점과 구조화된 추론 과정을 필요로 하는 사용자이다.

- User A (일반 사용자): 빠르게 검증된 결론을 얻고 싶은 사용자. 에이전트 설정을 최소화하고 토론 결과만 확인
- User B (고급 사용자): AI 설정과 토론 구조까지 직접 설계하고 싶은 사용자. 에이전트 역할, 모델, 추론 방식을 세밀하게 조정하고 토론 중 직접 질문을 주입

주요 Stakeholder는 세 그룹으로 구성된다. End User는 다양한 관점의 분석과 구조화된 결론을 통해 더 나은 의사결정을 내린다. AGORA Platform은 사용자 피드백을 기반으로 서비스를 지속 개선하고 확장한다. AI Model Providers(OpenAI, Google, Groq 등)는 자사 모델이 의사결정 지원 도구로 활용되는 새로운 사용 사례를 확보한다.

1.3 핵심 가치

AGORA는 네 가지 핵심 차별점을 가진다.

키워드	기능	설명
PERSONA	사용자 정의 페르소나	목적에 맞는 AI 에이전트 성향 및 역할을 직접 커스텀. 분석가, 비평가, 낙관론자 등 원하는 관점을 조합 가능
RAG	RAG 기반 지식 활용	사용자가 업로드한 문서를 기반으로 신뢰할 수 있는 근거 중심 분석 제공
DEBATE	에이전트 간 토론 구조	에이전트들이 서로의 주장을 교차 반박하고 검증하는 구조적 토론 진행
VERIFY	검증 중심 의사결정	상호 검증 과정을 통해 편향성을 최소화하고 균형 잡힌 최종 결론 도출

2. 기존 사례 분석 및 사업화 가능성

2.1 유사 제품/서비스

현재 AI 기반 의사결정 지원 도구는 빠르게 발전하고 있지만, 대부분의 서비스는 단일 에이전트 기반 구조에 머물러 있다.

서비스	강점	한계	AGORA 차별점
ChatGPT	빠른 응답, 넓은 지식 범위	단일 관점, sycophancy 편향, hallucination	다중 에이전트 교차 검증 구조
Perplexity AI	출처 기반 정보 신뢰성	관점 간 충돌·비교 없음, 의사결정 지원 미흡	구조화된 토론 및 의사결정 흐름
Multiple Chat AI	여러 모델에 동시 질문 가능	병렬 답변 나열에 그침, 에이전트 간 토론 없음	에이전트 간 반박·재반박 토론 구조
AutoGen / CrewAI	멀티 에이전트 프레임워크	개발자 전용, 일반 사용자 UX 없음	일반 사용자 대상 UI/UX 및 페르소나 커스텀

AGORA는 Multi-Agent 기반 구조화된 토론 시스템과 일반 사용자 대상 UI/UX를 결합한다는 점에서 기존 서비스들과 명확히 구분된다. Multiple Chat AI처럼 여러 모델의 답변을 나란히 보여주는 것이 아니라, 에이전트들이 서로 반박하고 검증하는 실질적인 토론 구조를 제공한다.

2.2 사업화 가능성

2.2.1 비즈니스 모델 정의

AGORA의 비즈니스 모델은 사용자, 플랫폼, AI 모델 제공자 간의 가치 교환 구조로 구성된다. 사용자는 질문/검증 요청을 AGORA 플랫폼에 전달하고 결과를 받는다. AGORA 플랫폼은 사용자 요청에 따라 AI Service Provider에 API를 호출하고 답변을 반환받아 토론 결과를 생성한다.

수익 구조는 세 가지 복합 모델로 구성된다.

- 구독 모델 (Subscription): 기본 기능 무료 제공 후 에이전트 수 확장, 고성능 모델 접근, RAG 통합 등 프리미엄 기능 유료화
- 사용량 기반 과금 (Pay-as-you-go): 대용량 토론 세션, 추가 토큰 사용량에 따른 과금
- B2B / 엔터프라이즈 (Enterprise): 팀 단위 협업 플랫폼, 조직용 의사결정 시스템, 맞춤형 에이전트 제공

2.2.2 시장규모 분석

AI 기반 의사결정 지원 시장은 빠르게 성장하고 있다. AGORA의 확장 로드맵은 3단계로 구성된다.

단계	시장	전략
Phase 1	개인 / 교육	대학생, 예비 창업자, 개인 사용자 대상 B2C 진입. Freemium으로 초기 사용자 확보
Phase 2	B2B SMB	스타트업, 소규모 기업 팀 단위 협업 플랜 제공. 팀별 에이전트 설정 공유 및 토론 기록 관리
Phase 3	엔터프라이즈	대기업 의사결정 지원 시스템, 맞춤형 AI 에이전트, 온프레미스 배포 옵션 제공

B2B 확장이 장기 수익의 핵심이다. 초기 B2C에서 사용자 기반과 제품 검증을 완료한 후, B2B SMB를 거쳐 엔터프라이즈로 확장하는 구조를 취한다.

2.2.3 상용화 시 예상 사업 규모

단계	총 사용자	유료 전환율	예상 월 매출
Phase 1 (론칭 후 6개월)	500명	8% ($9/월)	$360

단계	총 사용자	유료 전환율	예상 월 매출
Phase 2 (1년 후)	2,000명	10% ($9/월)	$1,800
Phase 3 (B2B 확장)	기업 고객 20곳	$50/월 팀 플랜	$1,000 추가

3. User Story Mapping

3.1 페르소나 정의

User A: 빠르게 결론을 얻고 싶은 일반 사용자

- 상황: 여러 선택지 중 하나를 결정해야 하지만 혼자 판단하기 어렵다.
- Pain Point: AI에게 물어보면 하나의 답변만 돌아오고, 여러 탭을 열어 각각 질문하는 것은 번거롭다.
- 기대: 에이전트 설정을 최소화하고 토론 결과와 최종 결론만 빠르게 확인하고 싶다.

User B: AI 설정과 토론 구조까지 직접 설계하고 싶은 고급 사용자

- 상황: 복잡한 전략적 의사결정을 내려야 하며, 다양한 LLM 모델을 실험하고 싶다.
- Pain Point: 기존 AI 도구는 에이전트 역할, 모델, 추론 방식을 세밀하게 조정할 수 없다.
- 기대: 에이전트별 모델과 역할을 직접 설정하고, 토론 중에 직접 추가 질문을 주입하여 토론 방향을 조정하고 싶다.

3.2 사용자 여정 (User Journey)

AGORA의 핵심 사용 흐름은 SET → DEBATE → RESULT의 3단계로 구성된다.

Step 01 (SET): 질문 입력 + 에이전트 설정

- 사용자가 질문 또는 의사결정 주제를 입력한다.
- 에이전트 수, 역할(페르소나), 모델, 추론 방식을 설정한다.
- 필요 시 개인 문서를 업로드하여 RAG 기반 지식 주입을 연동한다.

Step 02 (DEBATE): 다중 에이전트 교차 검증 토론

- Round 1 — Opening Statements: 각 에이전트가 독립적으로 초기 의견을 생성한다. Moderator AI가 Round 1 결과를 요약하고 주요 합의점/대립점을 제시한다.
- Round 2 — Cross Examination: Moderator가 가장 대립이 큰 에이전트 쌍을 선정한다. 선정된 쌍이 Challenge → Response → Rebuttal 순서로 교차 토론을 진행한다. User B는 이 단계에서 직접 추가 질문을 주입하여 토론 방향을 조정할 수 있다.

Step 03 (RESULT): 결과 도출

- Round 3 — Final Synthesis: Moderator AI가 전체 토론을 종합하여 최종 결론, 합의점, 미해결 쟁점, 의사결정 추천안을 구조화하여 제시한다.
- 사용자는 의사결정 근거 자료와 함께 결론을 확인하고, 필요 시 Follow-up 질문을 추가하거나 새로운 토론을 시작할 수 있다.

3.3 Epics & User Stories

Epic 1: 사용자 인증 및 세션 관리 (AUTH)

안전한 사용자 데이터 보호 및 중단 없는 연속적인 토론 세션 유지 환경 제공

- ● User Story 1.1: 나는 사용자로서, Agora 계정을 생성하고 로그인하고 싶다 → 내 토론 기록과 설정을 안전하게 관리하기 위해
- ● User Story 1.2: 나는 사용자로서, 새 Debate 세션을 생성하고 싶다 → 새로운 주제별로 토론을 분리해서 관리하기 위해
- ● User Story 1.3: 나는 사용자로서, 질문을 입력하고 검증받고 싶다 → 모든 에이전트가 같은 문제를 기준으로 토론하게 하기 위해

Epic 2: AI 에이전트 설정 (AGENT)

다양한 관점을 반영하기 위한 페르소나 커스텀 및 다중 AI 성향 조율 기능

- ● User Story 2.1: 나는 사용자로서, 새로운 AI 에이전트를 추가하고 싶다 → 한 토론 안에서 여러 관점을 동시에 비교하기 위해
- ● User Story 2.2: 나는 사용자로서, 에이전트의 역할과 관점 템플릿을 선택하고 싶다 → 분석가·비평가·낙관론자·리스크 분석가처럼 서로 다른 입장을 만들기 위해
- ● User Story 2.3: 나는 사용자로서, 각 에이전트의 모델·추론 방식·행동 특성을 세부 설정하고 싶다 → 역할은 같아도 다른 추론 전략을 비교하기 위해

Epic 3: RAG 시스템 (RAG)

사용자 데이터 및 외부 신뢰성 있는 문서를 기반으로 한 정확한 지식 활용 및 분석

- ● User Story 3.1: 나는 사용자로서, 개인 문서와 노트를 업로드하고 싶다 → AI 에이전트가 내 지식 베이스를 근거로 활용하게 하기 위해

- ● User Story 3.2: 나는 사용자로서, 업로드한 문서가 자동으로 파싱되고 인덱싱되길 원한다 → 토론 중 필요한 근거를 빠르게 검색하기 위해
- ● User Story 3.3: 나는 사용자로서, 특정 에이전트에게만 특정 지식 베이스 접근 권한을 주고 싶다 → 역할별로 참조 가능한 정보 범위를 다르게 설정하기 위해

● User Story 3.4: 나는 사용자로서, 에이전트가 근거를 검색하고 출처와 함께 주장에 연결하길 원한다 → 어떤 정보에 기반해 주장했는지 투명하게 확인하기 위해

Epic 4: Opening Statements

Debate 시작 및 각 에이전트의 초기 의견 생성 (User Journey Step 02 - Round 1)

● User Story 4.1: 나는 사용자로서, Start Debate 버튼으로 토론을 시작하고 싶다 → 설정한 질문과 에이전트 구성을 실제 토론 세션으로 실행하기 위해

● User Story 4.2: 나는 사용자로서, Round 1: Opening Statements를 보고 싶다 → 각 에이전트의 초기 입장을 서로 간섭 없이 비교하기 위해

● User Story 4.3: 나는 사용자로서, Moderator Panel에서 Round 1 요약을 보고 싶다 → 어떤 에이전트가 동의/대립하는지 쉽게 이해하기 위해

Epic 5: Cross Examination

핵심 충돌 쌍 선정 및 교차 반박 토론 진행 (User Journey Step 02 - Round 2)

● User Story 5.1: 나는 사용자로서, Start Cross Examination 액션으로 Round 2로 넘어가고 싶다 → 가장 중요한 쟁점 중심으로 심화 토론을 진행하기 위해

● User Story 5.2: 나는 사용자로서, 시스템이 핵심 충돌 쌍을 자동 선정하길 원한다 → 모든 에이전트가 동시에 싸우는 혼란 없이 가장 중요한 대립만 보고 싶기 때문에

● User Story 5.3: 나는 사용자로서, Round 2: Cross Examination thread을 보고 싶다 → 도전·응답·재반박 흐름으로 왜 의견이 갈리는지 명확히 이해하기 위해

● User Story 5.4: 나는 사용자로서, Moderator Panel에서 Round 2 요약을 보고 싶다 → 현재 충돌의 핵심과 아직 해결되지 않은 이슈를 빠르게 이해하기 위해

Epic 6: 사용자 질문 주입

토론 중 사용자가 직접 개입하여 방향 조정 (User Journey Step 02 - User Injection)

● User Story 6.1: 나는 사용자로서, 토론 후에 새로운 주장이나 의견을 추가하고 싶다 → AI 토론의 방향을 내가 직접 조정하기 위해

Epic 7: Final Synthesis

Moderator AI의 최종 종합 및 의사결정 추천안 도출 (User Journey Step 03 - RESULT)

● User Story 7.1: 나는 사용자로서, Generate Final Synthesis로 Round 3를 생성하고 싶다 → 교차토론 이후 수정된 최종 입장과 결론을 확인하기 위해

● User Story 7.2: 나는 사용자로서, Final Stance와 Debate Outcome Summary를 보고 싶다 → 토론의 최종 상태를 이해 가능한 결론 형태로 확인하기 위해
● User Story 7.3: 나는 사용자로서, Moderator의 최종 가이드를 보고 싶다 → 합의점·미해결점·의사결정 포인트를 한 번에 이해하기 위해
● User Story 7.4: 나는 사용자로서, Ask Follow-up Question 또는 Run Another Debate를 실행하고 싶다 → 현재 토론을 이어가거나 설정을 바꿔 새로운 토론을 시작하기 위해

Epic 8: 토론 시각화
주장·반박·근거 관계를 그래프로 표현하여 토론 구조 파악
● User Story 8.1: 나는 사용자로서, 주장·반박·근거 관계를 그래프로 보고 싶다 → 토론 구조와 핵심 충돌을 시각적으로 파악하기 위해

Epic 9: 토론 기록 관리
과거 토론 세션 저장 및 재열람
● User Story 9.1: 나는 사용자로서, 모든 토론 기록을 저장하고 다시 보고 싶다 → 이전 주장, 반박, 결론을 재검토하고 비교하기 위해
Epic 10: LLM 호출 로깅 및 모니터링

- ● User Story 10.1: 나는 개발자로서, 모든 LLM 호출 이력과 성능 지표를 추적하고 싶다 → 디버깅 및 성능 분석이 가능하도록

Epic 11: 오류 처리 및 진행 상태

- ● User Story 11.1: 나는 사용자로서, 시스템 오류 및 에이전트 타임아웃 상황에서도 명확한 안내를 받고 싶다 → 토론이 실패해도 상황을 이해하고 다음 행동을 알 수 있도록
- ● User Story 11.2: 나는 사용자로서, 토론 진행 중 각 라운드의 진행 상황을 실시간으로 확인하고 싶다 → 얼마나 남았는지 알고 기다릴 수 있도록

4. 설계 및 구현

4.1 기술 분석

4.1.1 기술 스택

영역	기술	선택 이유
Backend	FastAPI (Python)	비동기 처리로 다수 에이전트 LLM 요청 병렬 처리 가능. WebSocket 실시간 스트리밍 구현에 적합

영역	기술	선택 이유
Frontend	React (MUI)	컴포넌트 기반 구조로 복잡한 토론 UI(에이전트 설정, 토론 타임라인, Moderator 패널) 효율적 관리
Database	PostgreSQL + pgvector	관계형 데이터 구조 안정 관리(ACID 보장). pgvector로 벡터 검색 기반 RAG 동시 지원
LLM Provider	OpenRouter	GPT, Claude, Gemini 등 다양한 LLM에 단일 API로 접근. 에이전트별 모델 분리 및 비용 최적화 용이
Infra	GCP / Render	GCP는 DB 호스팅, Render는 Backend API 서버 배포
DevOps	Docker, GitHub, CI/CD	개발 환경 일관성 유지 및 자동 빌드·배포 파이프라인

4.1.2 개발 및 협업 환경

- 버전 관리: Git / GitHub (main / develop / feature 브랜치 전략)
- 코드 리뷰: Pull Request 기반 팀원 간 직접 리뷰
- 개발 환경: Docker 기반 로컬 개발 환경으로 일관성 유지
- 배포: Render (Backend), GCP (Database)
- CI/CD: GitHub Actions 기반 자동 빌드 및 배포
- API 테스트: Postman

4.1.3 핵심 기술 및 알고리듬 (Optional)

1) Multi-Agent Debate Orchestration

- Round 1: 여러 에이전트를 병렬 실행하여 독립적인 초기 의견 생성
- Round 2: 충돌 기반 토론 쌍 선택 알고리즘으로 대립이 가장 큰 에이전트 쌍을 우선 선정. Challenge → Response → Rebuttal 3단계 프롬프트 체인 실행
- 프롬프트 체이닝: 각 라운드 결과가 다음 라운드의 입력 컨텍스트로 누적되어 점진적으로 더 정교한 토론 결과 생성

2) WebSocket 기반 실시간 스트리밍

- 각 에이전트의 응답이 생성되는 즉시 WebSocket을 통해 프론트엔드에 전달
- round_started, round_completed, message_created, turn_completed, error 이벤트로 UI 실시간 갱신
- 연결 끊김 시 자동 재연결 로직

3) RAG (Retrieval-Augmented Generation)

- 사용자 업로드 문서를 청킹 후 OpenRouter Embedding API로 벡터 변환
- pgvector 유사도 검색으로 관련 문서 청크를 에이전트별 LLM 입력에 포함
- 에이전트별 문서 접근 권한 제어

4) Prompt Engineering 기반 역할 시스템

- 에이전트별 역할 정의(Analyst, Critic, Strategist 등) 및 preset 템플릿 제공
- 역할에 따른 사고 방식, 응답 스타일, reasoning mode 제어

4.1.4 하드웨어 구축 기술 (Optional)

● 해당 없음 (소프트웨어 전용 프로젝트)

4.2 시스템 설계

4.2.1 시스템 개요

4.2.2 소프트웨어 아키텍처

레이어	역할
API Layer	REST API 및 WebSocket 엔드포인트 제공, 클라이언트 통신 담당
Chat Service	ChatSession, ChatTurn, ChatAgent 생성 및 상태 관리
Chat Engine	전체 토론 실행 흐름 제어, 각 라운드 순차 실행
Round Manager	Round 1/2/3 세부 실행 로직 담당
Retrieval Service	문서 기반 벡터 검색, 관련 문서 청크 조회
LLM Service	OpenRouter API 호출, 응답 생성 및 스트리밍 처리
Data Layer	PostgreSQL ORM 기반 데이터 저장 및 조회

4.2.3 데이터 설계

테이블	설명
users	사용자 계정 정보
chat_sessions	Debate 세션. users와 연결
chat_agents	에이전트 설정 (역할, 모델, provider, temperature, reasoning style)
chat_turns	하나의 질문과 전체 토론 실행 단위

테이블	설명
rounds	ChatTurn 내부 라운드 단계 (Round 1/2/3)
messages	사용자 입력, 에이전트 응답, 비판, 최종 요약 등 모든 텍스트
llm_calls	LLM 호출 이력, provider, model, token 수, latency, 성공/실패
documents	업로드 문서 메타데이터
document_chunks	문서 청크 텍스트 및 pgvector embedding
subscriptions	사용자 플랜 정보, 결제 상태, 만료일 (상용화)
usage_logs	사용자별 토큰 사용량, API 비용 추적 (상용화)

4.2.4 API 정의

영역	엔드포인트	설명
Auth	POST /auth/register POST /auth/login POST /auth/refresh	회원가입, 로그인, 토큰 갱신
ChatSession	POST /chat-sessions GET /chat-sessions/{id} GET /chat-sessions/{id}/history	세션 생성, 조회, 전체 대화 기록
ChatAgent	POST /chat-sessions/{id}/agents PATCH /chat-agents/{id} PATCH /chat-agents/{id}/activate	에이전트 추가, 설정 수정, 활성화/비활성화
ChatTurn	POST /chat-sessions/{id}/turns/start POST /chat-turns/{id}/cancel	토론 시작, 취소
Round	POST /chat-turns/{id}/rounds/start GET /rounds/{id}/messages	라운드 실행, 메시지 조회
Documents	POST /api/v1/documents/upload GET /api/v1/documents DELETE /api/v1/documents/{id}	문서 업로드, 목록, 삭제
LLM	GET /llm/providers GET /llm/models	사용 가능한 provider·모델 목록

영역	엔드포인트	설명
WebSocket	WS /ws/chat-sessions/{id} WS /ws/chat-turns/{id}	실시간 이벤트 구독 (round_started, message_created 등)
Billing (상용화)	POST /billing/subscribe GET /billing/usage GET /billing/plans	구독 결제, 사용량 조회, 플랜 목록

5. 과제 수행 계획

프로젝트 과제 수행에 필요한 일정, 업무분장, 협업 방식, 성과 측정 방안을 기술한다.

5.1 일정계획

● 개발 일정은 번다운 차트로 대체하며 기획제안서에는 핵심 일정을 요약 제시함. 전체 개발 일정은 하단 표를 참조.

Story ID	User Story	핵심 Task
12.1.1	사용자로서, 무료/유료 플랜을 비교하고 업그레이드하고 싶다	[FE] 플랜 비교 페이지 및 업그레이드 CTA UI 구현
12.1.2	사용자로서, 구독을 결제하고 관리하고 싶다	[BE] Stripe 결제 연동 및 구독 생성·취소 API 구현 [DB] subscriptions 테이블 설계
12.1.3	사용자로서, 플랜별 기능 제한을 인식하고 싶다	[BE] 플랜별 에이전트 수 cap, 토론 횟수 제한 미들웨어 구현 [FE] 한도 초과 시 업그레이드 안내 UI
12.1.4	사용자로서, 결제 내역과 다음 결제일을 확인하고 싶다	[FE] 구독 관리 페이지 구현 [BE] GET /billing/subscription API

5.2 업무분장

● 개발 업무 분장: 번다운 차트 task 담당자 기준

- Asadbek: Backend (FastAPI, Debate Engine, LLM 연동, WebSocket)
- Bekhruzbek: Frontend (React, 토론 타임라인 UI, UX 고도화)
- Su Sandi Ko: 공통 (DB 설계, RAG 시스템, DevOps, QA)

● 비 개발 업무 분장: 기획 문서 작성 (전원), 발표 준비 (전원)

Story ID	User Story	핵심 Task
12.2.1	사용자로서, 내 토큰 사용량과 남은 한도를 확인하고 싶다	[FE] 사용량 대시보드 UI 구현 [BE] GET /billing/usage API
12.2.2	운영자로서, 사용자별 API 비용을 추적하고 싶다	[DB] usage_logs 테이블 설계 (토큰 수, 비용, 모델별 내역) [BE] LLM 호출 시 자동 usage 기록 로직

Story ID	User Story	핵심 Task
12.2.3	운영자로서, 사용량 한도 초과 시 자동으로 차단하고 싶다	[BE] 플랜별 월간 토큰 한도 초과 감지 및 차단 미들웨어 [FE] 한도 초과 에러 UI 및 업그레이드 유도

5.3 협업 및 데브옵스 자동화

- 코딩 컨벤션: PEP8 (Backend), ESLint/Prettier (Frontend)
- 코드 리뷰: Pull Request 기반 팀원 간 직접 리뷰, Squash Merge
- 깃 브랜치: main / develop / feature/* 브랜치 전략
- CI/CD: GitHub Actions 기반 자동 빌드 및 배포
- 데브옵스: Docker 로컬 환경, Render (Backend), GCP (DB)
- 테스트 자동화: pytest (Backend), Jest (Frontend)

Story ID	User Story	핵심 Task
12.3.1	운영자로서, 서비스 가동 상태와 에러율을 실시간 모니터링하고 싶다	[DevOps] 모니터링 도구 연동 (Sentry / Uptime Robot) [BE] 헬스체크 엔드포인트 구현 (GET /health)
12.3.2	운영자로서, 장애 발생 시 즉시 알림을 받고 싶다	[DevOps] 에러율 임계치 초과 시 Slack/이메일 알림 설정
12.3.3	운영자로서, API rate limiting을 적용하고 싶다	[BE] 사용자별 API 요청 rate limit 미들웨어 구현 (DDoS 방어)
12.3.4	운영자로서, production DB 백업을 자동화하고 싶다	[DevOps] GCP 자동 백업 스케줄 설정

5.4 성과 측정 방안

- 정량적 KPI: 토론 세션 완료율 ≥ 80%, API 평균 응답 시간 < 5초, 에러율 < 1%, 유료 플랜 전환율
- 정성적 KPI: 사용자 만족도 (NPS ≥ 30), 에이전트 응답 품질 사용자 평가, 의사결정 지원 유용성 피드백

Story ID	User Story	핵심 Task
12.4.1	사용자로서, 회원가입 후 AGORA 사용법을 안내받고 싶다	[FE] 온보딩 플로우 구현 (에이전트 설정 → 첫 토론 시작 가이드) [FE] 첫 방문 시 샘플 토론 세션 자동 생성

Story ID	User Story	핵심 Task
12.4.2	사용자로서, 토론 결과를 공유하고 싶다	[BE] 토론 결과 공유 링크 생성 API [FE] 공유 링크로 접근 시 읽기 전용 결과 뷰

단계	기간	주요 작업
플랜 및 결제 시스템	9주차	Stripe 결제 연동, subscriptions DB 설계, 플랜별 기능 제한 미들웨어, 플랜 비교 UI
토큰 추적 및 비용 관리	10주차	usage_logs DB 설계, 사용자별 토큰 추적, 한도 초과 차단 로직, 사용량 대시보드 UI
운영 안정성 및 모니터링	11주차	Sentry/Uptime Robot 연동, rate limiting 미들웨어, 헬스체크 API, DB 자동 백업 설정
온보딩 및 리텐션	12주차	온보딩 플로우, 샘플 토론 세션, 결과 공유 링크, 최종 QA 및 배포

※ 전체 개발 일정 요약 (번다운 차트 백로그 포함)

전체 개발 일정은 핵심 기능 구현(1~8주)과 상용화(9~12주)의 두 단계로 구성된다.

단계	기간	주요 작업
1단계: 인증 및 기본 시스템	1~2주차	회원가입·로그인, JWT 인증 및 보호 라우트, 사용자 DB 설계, 기본 API 라우팅, 인증 UI
2단계: 에이전트 관리	3주차	에이전트 CRUD, 역할·모델·reasoning 설정 구조, OpenRouter 연결, 활성화/비활성화 API, 에이전트 UI
3단계: WebSocket 인프라 및 세션	4주차	WebSocket 연결 관리, 실시간 이벤트 push, ChatTurn 생성·취소 API, 토론 상태 관리
4단계: 멀티 에이전트 토론 로직	5~6주차	Round 1 병렬 실행, Round 2 Cross-Critique 체인, 충돌 쌍 선택 알고리즘, Round 3 Final Synthesis, Moderator 요약
5단계: RAG 및 개인화	6~7주차	문서 업로드·파싱·청킹, 임베딩 생성·pgvector 인덱스, 벡터 검색 및 컨텍스트 주입, 에이전트별 접근 제어
6단계: UI/UX 고도화 및 오류 처리	7~8주차	토론 타임라인·Moderator 패널 UI, 에러 상태 UI, LLM 호출 로깅, 전체 흐름 UX 최적화
7단계: 테스트 및 배포	8주차	통합 테스트, 장애 시나리오 QA, 성능 최적화, GCP/Render 배포
8단계: 플랜 및 결제 시스템 (상용화)	9주차	Stripe 연동, 플랜 제한 미들웨어, 플랜 비교 UI, subscriptions DB
9단계: 비용 추적 및 모니터링 (상용화)	10~11주차	usage_logs, 토큰 한도 차단, Sentry 연동, rate limiting, DB 백업
10단계: 온보딩 및 최종 상용화 QA	12주차	온보딩 플로우, 결과 공유 링크, 최종 상용화 QA 및 발표 자료 정리